

SDSE Complete Notes

Software Design & Software Engineering

Topics Covered

- Software Lifecycle & Quality
- OOP (TypeScript)
- UML Diagrams
- ERD
- Design Patterns
- SOLID Principles
- SDLC Models
- Software Testing

Includes TypeScript code examples & MCQ practice questions

1	Software Engineering	3
1.1	Software Lifecycle	3
1.2	Quality Attributes	3
2	Object-Oriented Programming (OOP)	4
2.1	Core Concepts	4
2.1.1	Classes & Objects	4
2.1.2	Constructors	4
2.1.3	Invariants	4
2.2	OOP Mechanisms	5
2.2.1	Encapsulation	5
2.2.2	Abstraction	6
2.2.3	Inheritance	7
2.2.4	Polymorphism	7
2.3	Object Relationships & Design Concepts	8
2.3.1	Composition	8
2.3.2	DTO / Domain Object	9
2.3.3	Observability	9
3	Unified Modelling Language (UML)	10
3.1	Class Diagram	10
3.2	Class Relationships	10
3.3	Sequence Diagram	10
3.4	Use Case Diagram	10
4	Entity Relationship Diagram (ERD)	12
4.1	Components	12
4.2	Mapping Cardinalities	12
4.3	Participation Constraints	12
4.4	ERD Notations	12
5	Design Patterns	13
5.1	Creational Patterns	13
5.1.1	Singleton Pattern	13
5.1.2	Factory Pattern	13
5.2	Structural Patterns	14
5.2.1	Adapter Pattern	14
5.2.2	Composite Pattern	15
5.3	Behavioral Patterns	15
5.3.1	Observer Pattern	15
5.3.2	Strategy Pattern	16
6	SOLID Principles	18
6.1	Single Responsibility Principle (SRP)	18
6.2	Open/Closed Principle (OCP)	18
6.3	Liskov Substitution Principle (LSP)	19
6.4	Interface Segregation Principle (ISP)	19
6.5	Dependency Inversion Principle (DIP)	20

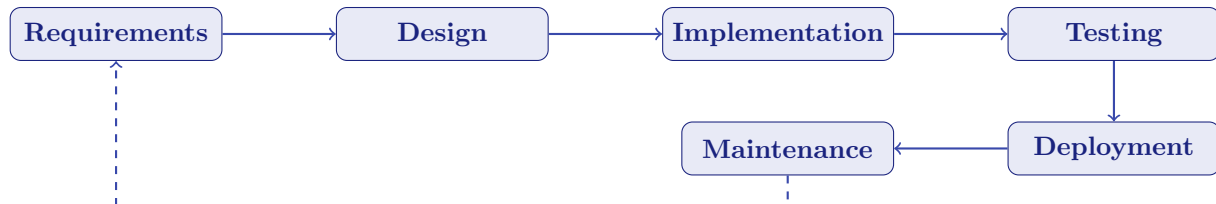
7	Software Development Life Cycle (SDLC)	22
7.1	Phases of SDLC	22
7.2	SDLC Models	22
7.2.1	Waterfall Model	22
7.2.2	Agile Model	22
8	Software Testing	24
8.1	What is Testing?	24
8.2	Types of Testing	24
8.2.1	Based on Scope	24
8.2.2	Black, White, and Grey Box Testing	25
8.2.3	Manual vs Automated Testing	25
8.2.4	Functional vs Non-Functional Testing	25
8.2.5	Testing Pyramid	25
	Quick Reference Card	27

1 1 Engineering

1.1 1Software Lifecycle

Definition

The **Software Lifecycle** describes the complete process of planning, creating, testing, deploying, and maintaining software. It provides a structured framework for managing complexity in software projects.



Key phases: **Requirements** (what to build) → **Design** (how to build) → **Implementation** (coding) → **Testing** (verification) → **Deployment** (release) → **Maintenance** (upkeep).

1.2 1Quality Attributes

Quality Attributes (ISO 25010)

Non-functional properties that describe *how well* software performs its functions.

Attribute	Description
Functionality	Correctness, completeness of features
Reliability	Fault tolerance, recoverability
Usability	Learnability, accessibility, aesthetics
Efficiency	Time behaviour, resource utilisation
Maintainability	Modularity, reusability, testability
Portability	Adaptability across environments
Security	Confidentiality, integrity, authenticity
Scalability	Handle growing load without degradation

MCQ Practice

Q1. Which quality attribute focuses on the ability of a system to handle increased load?

(A) Reliability (B) Portability (C) **Scalability** (D) Usability

Q2. The ISO 25010 quality model replaces which earlier standard?

(A) ISO 9001 (B) **ISO 9126** (C) IEEE 829 (D) CMMI

Answer: Q1: C, Q2: B

2 1

Object-

Oriented Programming (OOP)

2.1 1Core Concepts

2.1.1 1Classes & Objects

Classes & Objects

A **class** is a blueprint; an **object** is an instance of that blueprint carrying its own state.

TypeScript — Classes & Objects

```

1 class Animal {
2   name: string;
3   private age: number;
4
5   constructor(name: string, age: number) {
6     this.name = name;
7     this.age = age;
8   }
9
10  describe(): string {
11    return `${this.name} is ${this.age} years old`;
12  }
13 }
14
15 const dog = new Animal("Rex", 3);
16 console.log(dog.describe()); // Rex is 3 years old

```

2.1.2 1Constructors

A **constructor** is a special method that runs when an object is created. TypeScript supports **parameter properties** — shorthand for declaring and initialising in one step.

TypeScript — Constructor Shorthand

```

1 class Point {
2   // TypeScript parameter properties shorthand
3   constructor(
4     public x: number,
5     public y: number
6   ) {}
7
8   distanceTo(other: Point): number {
9     return Math.sqrt((this.x - other.x)**2 + (this.y - other.y)**2);
10  }
11 }
12
13 const p1 = new Point(0, 0);
14 const p2 = new Point(3, 4);
15 console.log(p1.distanceTo(p2)); // 5

```

2.1.3 1Invariants

Class Invariant

A **class invariant** is a condition that must always be *true* throughout the lifetime of an object. Constructors establish invariants; methods must preserve them.

TypeScript — Invariant via private setter

```
1 class BankAccount {
2   private _balance: number;
3
4   constructor(initialBalance: number) {
5     if (initialBalance < 0) throw new Error("Balance cannot be negative");
6     this._balance = initialBalance; // invariant: _balance >= 0
7   }
8
9   deposit(amount: number): void {
10    if (amount <= 0) throw new Error("Deposit must be positive");
11    this._balance += amount;
12  }
13
14  withdraw(amount: number): void {
15    if (amount > this._balance) throw new Error("Insufficient funds");
16    this._balance -= amount; // invariant maintained
17  }
18
19  get balance(): number { return this._balance; }
20 }
```

2.2 1OOP Mechanisms

2.2.1 1Encapsulation

Encapsulation

Bundling data and the methods that operate on it, while *hiding* internal state. Access modifiers: public, private, protected, readonly.

TypeScript — Getters & Setters

```
1 class Temperature {
2   private _celsius: number = 0;
3
4   get fahrenheit(): number {
5     return this._celsius * 9/5 + 32;
6   }
7
8   set celsius(value: number) {
9     if (value < -273.15) throw new RangeError("Below absolute zero!");
10    this._celsius = value;
11  }
12
13  get celsius(): number { return this._celsius; }
14 }
15
16 const t = new Temperature();
17 t.celsius = 100;
18 console.log(t.fahrenheit); // 212
```

2.2.2 1Abstraction

Abstraction

Exposing *what* something does without exposing *how* it does it. Achieved via **abstract classes** and **interfaces**.

TypeScript — Abstract Classes

```
1 abstract class Shape {
2     abstract area(): number;           // Must be implemented
3     abstract perimeter(): number;      // Must be implemented
4
5     describe(): string {
6         return `Area: ${this.area()}, Perimeter: ${this.perimeter()}`;
7     }
8 }
9
10 class Circle extends Shape {
11     constructor(private radius: number) { super(); }
12     area(): number { return Math.PI * this.radius ** 2; }
13     perimeter(): number { return 2 * Math.PI * this.radius; }
14 }
15
16 class Rectangle extends Shape {
17     constructor(private w: number, private h: number) { super(); }
18     area(): number { return this.w * this.h; }
19     perimeter(): number { return 2 * (this.w + this.h); }
20 }
```

TypeScript — Interfaces & Contracts

```
1 interface Printable {
2     print(): void;
3 }
4
5 interface Serializable {
6     serialize(): string;
7     deserialize(data: string): void;
8 }
9
10 // A class can implement multiple interfaces (unlike abstract classes)
11 class Document implements Printable, Serializable {
12     constructor(private content: string) {}
13
14     print(): void { console.log(this.content); }
15     serialize(): string { return JSON.stringify({ content: this.content }); }
16     deserialize(data: string): void {
17         this.content = JSON.parse(data).content;
18     }
19 }
```

Key Difference: Abstract Class vs Interface

	Abstract Class	Interface
Implementation	Can have concrete methods	No implementation (TS 4.x+: default methods)
Inheritance	Single inheritance	Multiple implements
Constructor	Yes	No
Access modifiers	Yes	No (all public)
Use case	Shared base behaviour	Contracts / capabilities

2.2.3 1Inheritance

Inheritance

A mechanism where a child class **extends** a parent class, acquiring its properties and methods while being able to override them.

TypeScript — Inheritance

```

1 class Vehicle {
2   constructor(public brand: string, public speed: number) {}
3
4   move(): string { return `${this.brand} moving at ${this.speed} km/h`;
5   }
6 }
7 class Car extends Vehicle {
8   constructor(brand: string, speed: number, public doors: number) {
9     super(brand, speed); // Call parent constructor
10  }
11
12  // Method override
13  move(): string {
14    return super.move() + ` with ${this.doors} doors`; // Call parent
15    // method
16  }
17 }
18 const car = new Car("Toyota", 120, 4);
19 console.log(car.move());
20 // Toyota moving at 120 km/h with 4 doors

```

2.2.4 1Polymorphism

Polymorphism

Many forms — the ability to treat objects of different classes through a common interface. TypeScript achieves this via **method overriding** (runtime/subtype polymorphism) and **method overloading** (compile-time).

TypeScript — Polymorphism

```

1 class Animal {
2   speak(): string { return "..."; }
3 }
4 class Dog extends Animal {
5   speak(): string { return "Woof!"; }

```

```

6  }
7  class Cat extends Animal {
8    speak(): string { return "Meow!"; }
9  }
10
11 // Polymorphic function      works for ANY Animal subtype
12 function makeNoise(animals: Animal[]): void {
13   animals.forEach(a => console.log(a.speak())); // Dynamic dispatch
14 }
15
16 makeNoise([new Dog(), new Cat(), new Dog()]);
17 // Woof! Meow! Woof!

```

MCQ Practice

Q3. Which OOP concept allows a subclass to provide a specific implementation of a method defined in its superclass?

(A) Encapsulation (B) Abstraction (C) **Method Overriding** (D) Composition

Q4. In TypeScript, which keyword is used to call the parent class constructor?

(A) `parent()` (B) `base()` (C) `this()` (D) `super()`

Q5. An interface in TypeScript cannot:

(A) Define method signatures (B) Be implemented by multiple classes
(C) Extend another interface (D) **Have a constructor**

Answer: Q3: C, Q4: D, Q5: D

2.3 1Object Relationships & Design Concepts

2.3.1 1Composition

Composition vs Inheritance

Composition = “has-a” relationship. Prefer composition over inheritance for flexibility — change behaviour at runtime by swapping components.

TypeScript — Composition

```

1  interface Engine { start(): string; }
2
3  class ElectricEngine implements Engine {
4    start(): string { return "Silently whirring..."; }
5  }
6  class PetrolEngine implements Engine {
7    start(): string { return "Vroom!"; }
8  }
9
10 class Car {
11   // Car HAS-A Engine (composition)
12   constructor(private engine: Engine) {}
13
14   startCar(): string { return this.engine.start(); }
15 }
16
17 const tesla = new Car(new ElectricEngine());
18 const bmw   = new Car(new PetrolEngine());
19 console.log(tesla.startCar()); // Silently whirring...

```

2.3.2 1DTO / Domain Object

Type	Description
DTO (Data Transfer Object)	Plain data container for moving data between layers. No business logic.
Domain Object	Rich object with both state <i>and</i> business logic. Models a real-world concept.

TypeScript — DTO vs Domain Object

```

1 // DTO      pure data, no logic
2 interface UserDTO {
3   id: number;
4   name: string;
5   email: string;
6 }
7
8 // Domain Object      state + behaviour
9 class User {
10   private loginCount: number = 0;
11
12   constructor(
13     public readonly id: number,
14     private name: string,
15     private email: string
16   ) {}
17
18   login(): void { this.loginCount++; }
19   isFrequentUser(): boolean { return this.loginCount > 10; }
20
21   toDTO(): UserDTO {
22     return { id: this.id, name: this.name, email: this.email };
23   }
24 }

```

2.3.3 1Observability

Observability

The ability to measure the *internal state* of a system from its external outputs. The three pillars:

Logs

Timestamped
records of events

Metrics

Numeric
measurements over
time

Traces

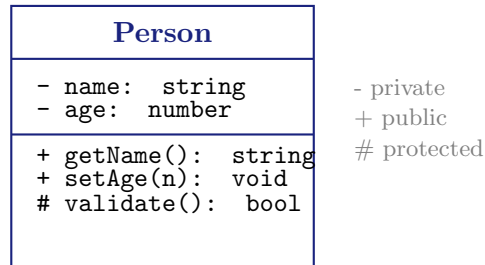
Request path
through services

3 1 Modelling Language (UML)

3.1 1Class Diagram

Class Diagram

Static structural diagram showing classes, their attributes, methods, and relationships.



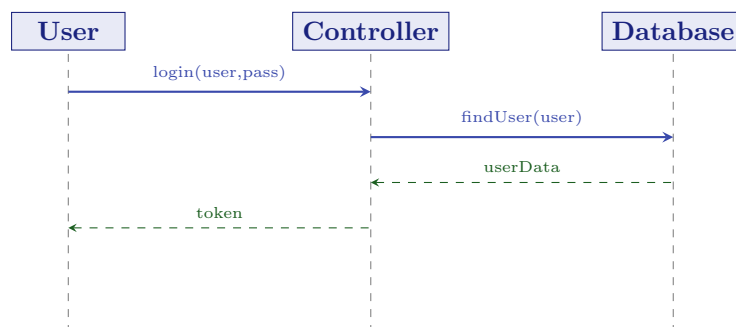
3.2 1Class Relationships

Relationship	Notation	Meaning
Association	--->	General use relationship
Aggregation	---◇	“has-a”, part can exist independently
Composition	---◆	Strong “has-a”, part cannot exist alone
Inheritance	---△	“is-a”, extends parent
Realization	- - -△	Implements interface
Dependency	- - ->	Uses temporarily

3.3 1Sequence Diagram

Sequence Diagram

Shows *interactions between objects over time*, ordered vertically from top to bottom. Key elements: **lifelines**, **activation bars**, **messages (sync/async/return)**.



3.4 1Use Case Diagram

Use Case Diagram

Shows *functional requirements* — what the system does and who interacts with it. Key elements: **Actors** (stick figures), **Use Cases** (ellipses), **System boundary** (rectangle).

Relationships: **Association** (actor–use case), **<<include>>** (mandatory sub-function), **<<extend>>** (optional/conditional behaviour), **Generalization** (actor/use case hierarchy).

MCQ Practice

Q6. In a UML class diagram, a filled diamond represents:

- (A) Association (B) Realization (C) Aggregation (D) **Composition**

Q7. Which UML diagram shows the order of messages exchanged between objects?

- (A) Class Diagram (B) Use Case Diagram (C) **Sequence Diagram** (D) ERD

Q8. The <<include>> relationship in a Use Case diagram means:

- (A) The base use case optionally uses the included case
(B) **The base use case always calls the included use case**
(C) One actor extends another actor
(D) A use case extends the system boundary

Answer: Q6: D, Q7: C, Q8: B

4 1

Entity

Relationship Diagram (ERD)

4.1 1Components

Component	Description / Notation
Entity	Rectangle — represents a real-world object (e.g., Student)
Weak Entity	Double rectangle — exists only with a parent entity
Attribute	Ellipse — property of an entity
Key Attribute	Underlined ellipse — uniquely identifies an entity
Multi-valued Attr.	Double ellipse — can have multiple values
Derived Attribute	Dashed ellipse — computed from other attributes
Relationship	Diamond — links two or more entities
Weak Relationship	Double diamond — links a weak entity to its owner

4.2 1Mapping Cardinalities

Type	Example	Meaning
1:1	Person — Passport	Each person has exactly one passport
1:N	Teacher — Students	One teacher teaches many students
M:1	Many students — One department	Many map to one
M:N	Students — Courses	Many students take many courses

4.3 1Participation Constraints

Participation Constraints

- **Total Participation** (double line): Every entity instance *must* participate in the relationship. E.g., every employee *must* work for a department.
- **Partial Participation** (single line): Entity instance *may or may not* participate. E.g., some employees may manage a department (but not all).

4.4 1ERD Notations

Notation	Chen	Crow's Foot
One (mandatory)		(two lines + perpendicular)
One (optional)	with 0	o
Many (mandatory)	N or M	Crow's foot ⊣
Many (optional)	N with 0	o ⊣

MCQ Practice

Q9. In an ERD, a double ellipse represents a:

- (A) Key attribute (B) Derived attribute (C) **Multi-valued attribute** (D) Weak entity

Q10. Total participation is represented in Chen notation by:

- (A) A single line (B) A dashed line (C) **A double line** (D) An arrow

Q11. In Crow's Foot notation, a "crow's foot" symbol represents:

- (A) Exactly one (B) Zero or one (C) **Many** (D) Zero

Answer: Q9: C, Q10: C, Q11: C

5 1 Patterns

Design

What are Design Patterns?

Reusable solutions to commonly occurring software design problems. Introduced by the “Gang of Four” (GoF). Three categories: **Creational**, **Structural**, **Behavioral**.

5.1 1Creational Patterns

5.1.1 1Singleton Pattern

Singleton

Ensures a class has *only one instance* and provides a *global access point* to it.

TypeScript — Singleton (Thread-safe lazy)

```
1 class DatabaseConnection {
2   private static instance: DatabaseConnection | null = null;
3   private connectionCount: number = 0;
4
5   // Private constructor prevents direct instantiation
6   private constructor(private url: string) {}
7
8   // Global access point
9   static getInstance(url: string): DatabaseConnection {
10    if (!DatabaseConnection.instance) {
11      DatabaseConnection.instance = new DatabaseConnection(url);
12    }
13    return DatabaseConnection.instance;
14  }
15
16  connect(): void {
17    this.connectionCount++;
18    console.log('Connected. Total: ${this.connectionCount}');
19  }
20 }
21
22 const db1 = DatabaseConnection.getInstance("db://localhost");
23 const db2 = DatabaseConnection.getInstance("db://localhost");
24 console.log(db1 === db2); // true      same instance
```

When to use Singleton

- Configuration manager
- Logger
- Database connection pool
- Thread pool

Caution: Can make testing difficult — prefer dependency injection where possible.

5.1.2 1Factory Pattern

Factory Pattern

Defines an interface for creating objects but lets subclasses decide which class to instantiate. *Decouples object creation from usage.*

TypeScript — Factory Pattern

```

1 interface Notification {
2     send(message: string): void;
3 }
4
5 class EmailNotification implements Notification {
6     send(message: string): void { console.log('Email: ${message}'); }
7 }
8 class SMSNotification implements Notification {
9     send(message: string): void { console.log('SMS: ${message}'); }
10 }
11 class PushNotification implements Notification {
12     send(message: string): void { console.log('Push: ${message}'); }
13 }
14
15 // Factory      decouples creation logic
16 class NotificationFactory {
17     static create(type: "email" | "sms" | "push"): Notification {
18         const map = {
19             email: EmailNotification,
20             sms: SMSNotification,
21             push: PushNotification,
22         };
23         return new map[type]();
24     }
25 }
26
27 const n = NotificationFactory.create("sms");
28 n.send("Your OTP is 1234"); // SMS: Your OTP is 1234

```

5.2 1Structural Patterns

5.2.1 1Adapter Pattern

Adapter Pattern

Converts the interface of a class into another interface that clients expect. Useful for *wrapping third-party APIs* for compatibility.

TypeScript — Adapter Pattern

```

1 // Our expected interface
2 interface Logger {
3     log(level: "info"|"warn"|"error", msg: string): void;
4 }
5
6 // Third-party library with incompatible interface
7 class LegacyLogger {
8     writeInfo(msg: string) { console.log('[INFO] ${msg}'); }
9     writeError(msg: string) { console.error('[ERR] ${msg}'); }
10 }
11
12 // Adapter wraps LegacyLogger to match our Logger interface
13 class LoggerAdapter implements Logger {
14     constructor(private legacy: LegacyLogger) {}
15
16     log(level: "info"|"warn"|"error", msg: string): void {
17         if (level === "info") this.legacy.writeInfo(msg);

```

```

18     if (level === "error") this.legacy.writeError(msg);
19     if (level === "warn")  this.legacy.writeInfo('WARN: ${msg}');
20   }
21 }
22
23 const logger: Logger = new LoggerAdapter(new LegacyLogger());
24 logger.log("info", "Application started");

```

5.2.2 Composite Pattern

Composite Pattern

Composes objects into *tree structures* to represent part-whole hierarchies. Clients treat individual objects and composites uniformly.

TypeScript — Composite (File System)

```

1 interface FileSystemItem {
2   getName(): string;
3   getSize(): number;
4 }
5
6 class File implements FileSystemItem {
7   constructor(private name: string, private size: number) {}
8   getName(): string { return this.name; }
9   getSize(): number { return this.size; }
10 }
11
12 class Folder implements FileSystemItem {
13   private children: FileSystemItem[] = [];
14   constructor(private name: string) {}
15
16   add(item: FileSystemItem): void { this.children.push(item); }
17   getName(): string { return this.name; }
18   // Recursive size computation      treats all children uniformly
19   getSize(): number { return this.children.reduce((s,c) => s+c.getSize()
20     , 0); }
21 }
22
23 const root = new Folder("root");
24 const docs = new Folder("docs");
25 docs.add(new File("resume.pdf", 120));
26 docs.add(new File("cover.docx", 45));
27 root.add(docs);
28 root.add(new File("readme.txt", 5));
29 console.log(root.getSize()); // 170

```

5.3 Behavioral Patterns

5.3.1 Observer Pattern

Observer Pattern

Defines a one-to-many dependency so that when one object changes state, all its dependents are notified and updated automatically. Used for *decoupled event systems*.

TypeScript — Observer Pattern

```

1 interface Observer {
2   update(event: string, data: unknown): void;
3 }
4
5 class EventEmitter {
6   private listeners = new Map<string, Observer[]>();
7
8   subscribe(event: string, observer: Observer): void {
9     if (!this.listeners.has(event)) this.listeners.set(event, []);
10    this.listeners.get(event)!.push(observer);
11  }
12
13  emit(event: string, data: unknown): void {
14    this.listeners.get(event)?.forEach(o => o.update(event, data));
15  }
16 }
17
18 class EmailService implements Observer {
19   update(event: string, data: unknown): void {
20     console.log('Email service reacted to ${event}:', data);
21   }
22 }
23
24 const emitter = new EventEmitter();
25 emitter.subscribe("userCreated", new EmailService());
26 emitter.emit("userCreated", { id: 1, email: "a@b.com" });

```

5.3.2 Strategy Pattern

Strategy Pattern

Defines a family of algorithms, encapsulates each one, and makes them interchangeable. Enables *runtime decision-making* — swap algorithm without changing the client.

TypeScript — Strategy Pattern

```

1 interface SortStrategy {
2   sort(data: number[]): number[];
3 }
4
5 class BubbleSort implements SortStrategy {
6   sort(data: number[]): number[] {
7     const arr = [...data];
8     for (let i = 0; i < arr.length; i++)
9       for (let j = 0; j < arr.length - i - 1; j++)
10        if (arr[j] > arr[j+1]) [arr[j], arr[j+1]] = [arr[j+1], arr[j]];
11     return arr;
12   }
13 }
14
15 class QuickSort implements SortStrategy {
16   sort(data: number[]): number[] {
17     if (data.length <= 1) return data;
18     const pivot = data[0];
19     const left = data.slice(1).filter(x => x <= pivot);
20     const right = data.slice(1).filter(x => x > pivot);
21     return [...this.sort(left), pivot, ...this.sort(right)];

```

```

22     }
23 }
24
25 class Sorter {
26     constructor(private strategy: SortStrategy) {}
27     setStrategy(s: SortStrategy) { this.strategy = s; } // swap at runtime
28     sort(data: number[]): number[] { return this.strategy.sort(data); }
29 }
30
31 const sorter = new Sorter(new BubbleSort());
32 console.log(sorter.sort([5,2,8,1])); // [1,2,5,8]
33 sorter.setStrategy(new QuickSort()); // Swap strategy
34 console.log(sorter.sort([5,2,8,1])); // [1,2,5,8]

```

Strategy vs Template Method

	Strategy	Template Method
Mechanism	Composition (interface)	Inheritance (abstract class)
Algorithm	Entirely swappable	Fixed skeleton, steps overridable
Flexibility	High (runtime swap)	Lower (compile-time)
Coupling	Loose	Tighter

MCQ Practice

- Q12.** Which pattern ensures a class has only one instance?
 (A) Factory (B) Adapter (C) Observer (D) **Singleton**
- Q13.** The Observer pattern implements which type of dependency?
 (A) Many-to-one (B) **One-to-many** (C) One-to-one (D) Many-to-many
- Q14.** Which structural pattern is used to make incompatible interfaces work together?
 (A) Composite (B) **Adapter** (C) Strategy (D) Singleton
- Q15.** The Strategy pattern changes algorithm at:
 (A) Compile time (B) Design time (C) **Runtime** (D) Test time
- Answer: Q12: D, Q13: B, Q14: B, Q15: C**

6 1 Principles

SOLID

SOLID Overview

Five principles by Robert C. Martin (“Uncle Bob”) for writing maintainable OO software.

- S** Single Responsibility Principle
- O** Open/Closed Principle
- L** Liskov Substitution Principle
- I** Interface Segregation Principle
- D** Dependency Inversion Principle

6.1 1Single Responsibility Principle (SRP)

SRP

“A class should have only one reason to change.” Each class should do one thing and do it well.

TypeScript — SRP Violation & Fix

```

1 //      VIOLATION: One class doing too much
2 class User {
3   constructor(public name: string, public email: string) {}
4   save(): void { /* DB logic */ }           // persistence concern
5   sendEmail(): void { /* Email logic */ }    // notification concern
6   validate(): void { /* validation */ }      // validation concern
7 }
8
9 //      CORRECT: Separate responsibilities
10 class User {
11   constructor(public name: string, public email: string) {}
12 }
13 class UserRepository {
14   save(user: User): void { /* DB logic only */ }
15 }
16 class UserEmailService {
17   sendWelcome(user: User): void { /* Email logic only */ }
18 }
19 class UserValidator {
20   validate(user: User): boolean { return !!user.email.includes("@"); }
21 }

```

6.2 1Open/Closed Principle (OCP)

OCP

“Software entities should be open for extension but closed for modification.” Add new features by adding new code, not changing existing code.

TypeScript — OCP via Strategy

```

1 //      VIOLATION      adding discount type requires modifying this class
2 class DiscountService {
3   calculate(type: string, price: number): number {
4     if (type === "student") return price * 0.8;
5     if (type === "senior")   return price * 0.7;
6     // Must edit to add more      violates OCP

```

```

7     return price;
8   }
9 }
10
11 //      OCP COMPLIANT      extend by adding new class, never modify
    existing
12 interface DiscountStrategy { apply(price: number): number; }
13 class StudentDiscount implements DiscountStrategy {
14   apply(price: number): number { return price * 0.8; }
15 }
16 class SeniorDiscount implements DiscountStrategy {
17   apply(price: number): number { return price * 0.7; }
18 }
19 // Adding new type = new class, zero existing code changes
20 class BlackFridayDiscount implements DiscountStrategy {
21   apply(price: number): number { return price * 0.5; }
22 }

```

6.3 1Liskov Substitution Principle (LSP)

LSP

“Objects of a superclass should be replaceable with objects of its subclasses without altering program correctness.” Subtypes must honour the parent’s contract.

TypeScript — LSP Violation & Fix

```

1 //      VIOLATION      Square breaks Rectangle's contract
2 class Rectangle {
3   constructor(public width: number, public height: number) {}
4   area(): number { return this.width * this.height; }
5 }
6 class Square extends Rectangle {
7   // Forces equal sides      violates Liskov since setting width !=
    height
8   set width(v: number) { super.width = v; super.height = v; } // BAD
9 }
10
11 //      FIX      Use a common abstraction
12 interface Shape { area(): number; }
13 class Rectangle implements Shape {
14   constructor(private w: number, private h: number) {}
15   area(): number { return this.w * this.h; }
16 }
17 class Square implements Shape {
18   constructor(private side: number) {}
19   area(): number { return this.side ** 2; }
20 }

```

6.4 1Interface Segregation Principle (ISP)

ISP

“Clients should not be forced to depend on interfaces they do not use.” Split fat interfaces into smaller, specific ones.

TypeScript — ISP

```

1 //      FAT interface      Robot doesn't eat but is forced to implement
   eat()
2 interface Worker {
3   work(): void;
4   eat(): void;
5   sleep(): void;
6 }
7
8 //      SEGREGATED      implement only what you need
9 interface Workable { work(): void; }
10 interface Eatable { eat(): void; }
11 interface Sleepable { sleep(): void; }
12
13 class HumanWorker implements Workable, Eatable, Sleepable {
14   work(): void { console.log("Working..."); }
15   eat(): void { console.log("Eating lunch..."); }
16   sleep(): void { console.log("Sleeping..."); }
17 }
18
19 class RobotWorker implements Workable { // Only what it needs
20   work(): void { console.log("Robot working 24/7..."); }
21 }

```

6.5 1Dependency Inversion Principle (DIP)

DIP

"High-level modules should not depend on low-level modules. Both should depend on abstractions."

TypeScript — DIP via Dependency Injection

```

1 //      VIOLATION      High-level depends on low-level concrete class
2 class MySQLDatabase { query(sql: string): string[] { return []; } }
3 class UserService {
4   private db = new MySQLDatabase(); // Hard dependency      WRONG
5   getUsers(): string[] { return this.db.query("SELECT * FROM users"); }
6 }
7
8 //      DIP COMPLIANT      both depend on abstraction
9 interface Database {
10   query(sql: string): string[];
11 }
12 class MySQLDatabase implements Database {
13   query(sql: string): string[] { return ['MySQL result for: ${sql}']; }
14 }
15 class PostgresDatabase implements Database {
16   query(sql: string): string[] { return ['Postgres result for: ${sql}']; }
17 }
18 class UserService {
19   constructor(private db: Database) {} // Depend on abstraction
20
21   getUsers(): string[] { return this.db.query("SELECT * FROM users"); }
22 }
23
24 // Inject dependency from outside
25 const service = new UserService(new PostgresDatabase());

```

MCQ Practice

Q16. “A class should have only one reason to change” describes:

(A) OCP (B) LSP (C) DIP (D) **SRP**

Q17. A Square extending Rectangle violates which SOLID principle?

(A) SRP (B) OCP (C) **LSP** (D) ISP

Q18. DIP states that high-level modules should depend on:

(A) Low-level modules (B) Concrete classes (C) **Abstractions** (D) Singletons

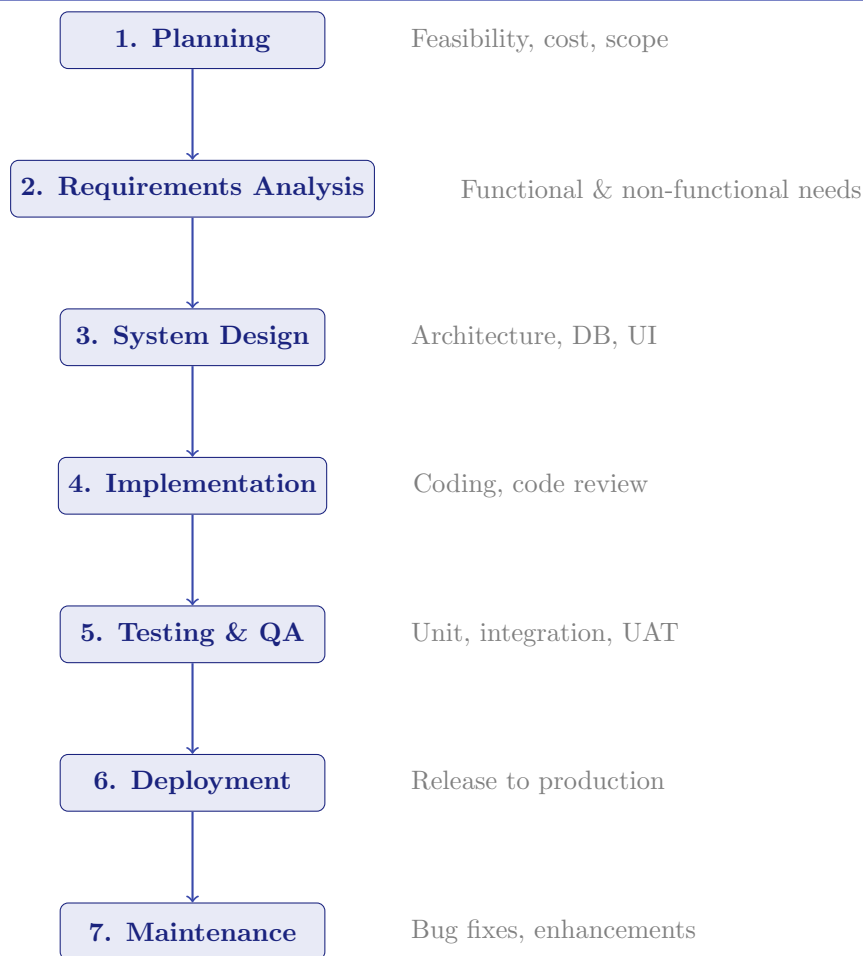
Q19. Splitting a “fat” interface into smaller ones follows:

(A) SRP (B) OCP (C) LSP (D) **ISP**

Answer: Q16: D, Q17: C, Q18: C, Q19: D

7.1 Development Life Cycle (SDLC)

7.1.1 Phases of SDLC



7.2 SDLC Models

7.2.1 Waterfall Model

Waterfall Model

Sequential, phase-by-phase development. Each phase must be *completed before the next begins*. Minimal backtracking.

Advantages Well-documented, simple to manage, works for stable requirements

Disadvantages Late delivery, poor for changing requirements, testing at the end

Best for: Projects with fixed, well-understood requirements (government contracts, hardware systems).

7.2.2 Agile Model

Agile Model

Iterative, incremental development in short cycles called **sprints** (1–4 weeks). Welcomes changing requirements. Values working software over documentation.

Agile Value	Details
Individuals & interactions	Over processes and tools
Working software	Over comprehensive documentation
Customer collaboration	Over contract negotiation
Responding to change	Over following a plan

Scrum artefacts: Product Backlog, Sprint Backlog, Increment.

Scrum ceremonies: Sprint Planning, Daily Standup, Sprint Review, Retrospective.

	Waterfall	Agile
Flexibility	Low	High
Delivery	End of project	Every sprint
Documentation	Heavy	Lightweight
Customer involvement	Start & end	Continuous
Risk	Late discovery	Early detection

MCQ Practice

Q20. In Waterfall, when does testing typically occur?

(A) Beginning (B) Throughout (C) **After implementation** (D) During design

Q21. What is a “sprint” in Agile/Scrum?

(A) A type of testing (B) **A fixed time-boxed iteration** (C) A code review (D) A design phase

Q22. Which SDLC model is best for projects with unclear or evolving requirements?

(A) Waterfall (B) V-Model (C) **Agile** (D) Spiral

Answer: Q20: C, Q21: B, Q22: C

8 1

Software

Testing

8.1 1What is Testing?

Software Testing

The process of *evaluating* a software system to detect differences between expected and actual outcomes. Goal: find bugs *before* users do.

Verification vs Validation:

- **Verification:** Are we building the product *right*? (meets specifications)
- **Validation:** Are we building the *right* product? (meets user needs)

8.2 1Types of Testing

8.2.1 1Based on Scope

Type	What is tested	Who tests
Unit Testing	Individual functions/classes in isolation	Developer
Integration Testing	Interaction between modules/services	Developer / QA
E2E Testing	Full user workflow from UI to DB	QA
Acceptance Testing	Business requirements & user stories	Client / QA

TypeScript — Unit Test (Jest)

```

1 // Function to test
2 function add(a: number, b: number): number { return a + b; }
3
4 // Jest unit test
5 describe("add function", () => {
6   it("should return correct sum", () => {
7     expect(add(2, 3)).toBe(5);
8   });
9
10  it("should handle negative numbers", () => {
11    expect(add(-1, 1)).toBe(0);
12  });
13
14  it("should handle zero", () => {
15    expect(add(0, 0)).toBe(0);
16  });
17 });

```

TypeScript — Integration Test

```

1 // Testing that UserService + Database work together
2 class UserRepository {
3   private users: Map<number, string> = new Map();
4   save(id: number, name: string): void { this.users.set(id, name); }
5   find(id: number): string | undefined { return this.users.get(id); }
6 }
7
8 class UserService {
9   constructor(private repo: UserRepository) {}
10  createUser(id: number, name: string): void { this.repo.save(id, name); }
11  getUser(id: number): string | undefined { return this.repo.find(id); }
12 }

```

```

13
14 // Integration test      tests both together
15 describe("UserService integration", () => {
16   it("should persist and retrieve user", () => {
17     const repo = new UserRepository();
18     const service = new UserService(repo);
19     service.createUser(1, "Alice");
20     expect(service.getUser(1)).toBe("Alice");
21   });
22 });

```

8.2.2 1Black, White, and Grey Box Testing

Type	Tester's knowledge	Focus
Black Box	No internal knowledge	Inputs/outputs only
White Box	Full code/internal knowledge	Logic, paths, coverage
Grey Box	Partial knowledge	Mix of both

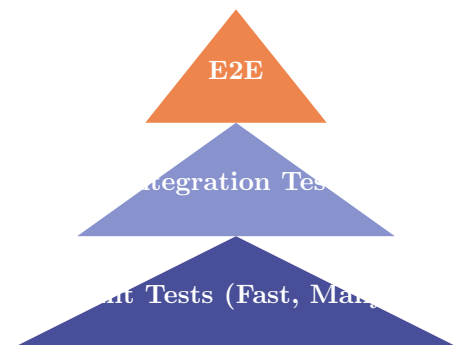
8.2.3 1Manual vs Automated Testing

	Manual	Automated
Speed	Slow	Fast
Cost	Low initial, high long-term	High initial, low long-term
Reliability	Human error possible	Consistent
Best for	Exploratory, UX, ad-hoc	Regression, CI/CD pipelines

8.2.4 1Functional vs Non-Functional Testing

	Functional	Non-Functional
Tests	What the system does	How well it performs
Examples	Login, payment, search	Load, stress, security, usability
Measurable	Feature works / doesn't	Metrics: latency, throughput

8.2.5 1Testing Pyramid



Testing Best Practices

- Write tests alongside code (TDD — Test Driven Development)
- Aim for high unit test coverage (>80%)
- Tests should be **F.I.R.S.T**: Fast, Isolated, Repeatable, Self-validating, Timely
- Use mocks/stubs for external dependencies in unit tests
- Automate regression tests in CI/CD pipeline

MCQ Practice

Q23. Which testing type requires full knowledge of the internal code structure?

- (A) Black Box (B) Grey Box (C) **White Box** (D) Acceptance

Q24. E2E (End-to-End) testing tests:

- (A) A single function (B) Two modules together
(C) **The complete user workflow** (D) Database queries only

Q25. Performance testing is an example of:

- (A) Functional testing (B) Unit testing (C) **Non-functional testing** (D) White box testing

Q26. TDD stands for:

- (A) Total Design Document (B) Test Driven Deployment
(C) **Test Driven Development** (D) Technical Design Document

Q27. According to the testing pyramid, which tests should be most numerous?

- (A) E2E Tests (B) Integration Tests (C) **Unit Tests** (D) Acceptance Tests

Answer: Q23: C, Q24: C, Q25: C, Q26: C, Q27: C

8 1 Reference Card

OOP Access Modifiers (TypeScript)

<code>public</code>	Accessible everywhere
<code>private</code>	Class only
<code>protected</code>	Class + subclasses
<code>readonly</code>	Cannot be reassigned
<code>#field</code>	Hard private (ES2022)

Design Patterns Summary

Singleton	One instance
Factory	Object creation
Adapter	Interface bridge
Composite	Tree structure
Observer	Event system
Strategy	Swap algorithms

SOLID Mnemonics

S	1 class = 1 job
O	Extend, don't modify
L	Subtypes = swappable
I	Small interfaces
D	Depend on abstractions

Testing Types

Unit	1 function/class
Integration	Module interaction
E2E	Full workflow
Acceptance	User stories
Black box	No code access
White box	Full code access

All MCQ Answers

Q1: C	Q2: B	Q3: C	Q10: C	Q11: C	Q12: D	Q19: D	Q20: C	Q21: B
Q4: D	Q5: D	Q6: D	Q13: B	Q14: B	Q15: C	Q22: C	Q23: C	Q24: C
Q7: C	Q8: B	Q9: C	Q16: D	Q17: C	Q18: C	Q25: C	Q26: C	Q27: C